Quicksort

Part II Sorting and Order Statistics

Scope and prerequisites

Partitioning establishes relations between regions and a pivot; recursion is still needed to sort. Follow Lomuto's invariant through every numerical state, then distinguish worst-case time, expected comparisons over internal randomness, and stack depth, including duplicate-key degeneration. Prerequisites are array partitioning, recursion, and expectation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

- Implement partition with its invariant.
- Distinguish worst, best, and expected Quicksort costs.
- Analyze random pivots and adversarial input order.

Sections 7.1–7.4 cover the algorithm, performance, randomization, and analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.1 Description of quicksort

Choose a pivot, separate other keys by their relation to it, and recursively sort both sides. Partition alone does not order keys within a side. Use 1-based Lomuto partition with pivot at the final position r.

```
quicksort(A,p,r):
    if p>=r: return
    q=partition(A,p,r)
    quicksort(A,p,q-1)
    quicksort(A,q+1,r)
partition(A,p,r):
    pivot=A[r]; i=p-1
    for j=p..r-1:
        if A[j]<=pivot:
            i++; swap(A[i],A[j])
    swap(A[i+1],A[r])
    return i+1
```

Empty and singleton intervals are bases. Both child intervals exclude q and strictly shrink. Including q again can cause infinite recursion when the pivot reaches an endpoint.

Loop boundaries: i ends the small region; j is next

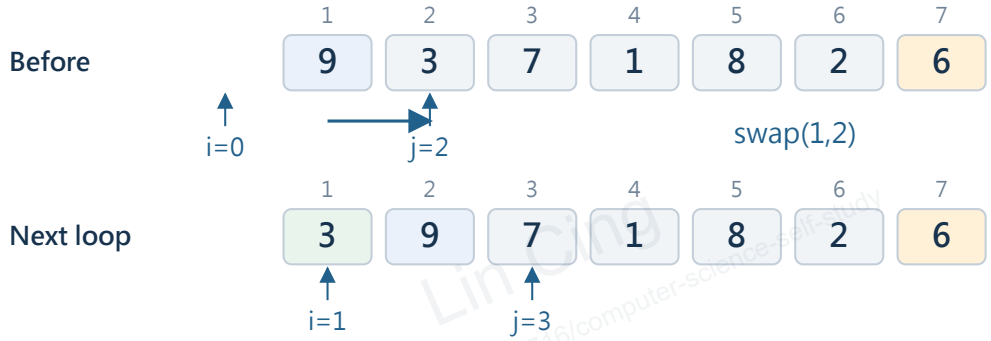
The pivot stays at A[r]. Use **1-based indices and the inclusive interval A[p..r]**. The boundary i=p−1 means that the ≤ region is empty.

Region	Known relation	Initially i=p−1, j=p
A[p..i]	≤ pivot	Empty
A[i+1..j−1]	> pivot	Empty
A[j..r−1]	Unclassified	All non-pivot keys
A[r]	Pivot	Fixed until the final exchange

Diagram key: green ≤, blue >, gray unknown, orange pivot. Both transition diagrams use [9,3,7,1,8,2,6] with pivot 6.

A small key: exchange, then advance both boundaries

3 ≤ 6: advance i, then exchange A[1] and A[2]



3 joins ≤; displaced 9 stays in >. Pivot 6 remains at index 7.

For A[2]=3, advance i to 1 and exchange A[1],A[2]. The key 3 joins ≤; displaced 9 stays in >. Finally advance j. In general i may equal j, giving a harmless self-exchange.

A large key: leave values fixed and advance j

7 > 6: no exchange; keep i and advance j

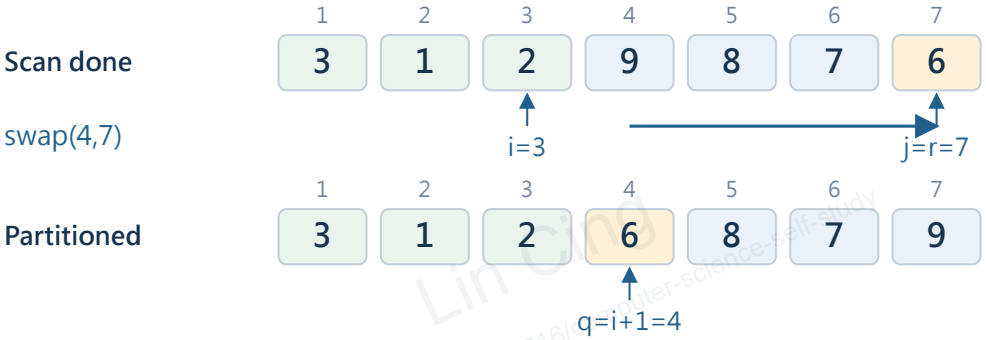


7 moves from unknown to >. Values stay fixed; the boundary moves.

Since A[3]=7>6, leave i and the array unchanged. Advancing j adds 7 to the > region. **The values stay fixed; our classification changes.**

The final exchange establishes the pivot position q

Only after scanning: place the pivot between the two regions



≤6: [3,1,2] | pivot 6 | >6: [8,7,9]; the sides remain unsorted.

The unknown length r−j decreases once per loop, proving termination. At j=r, exchange A[i+1],A[r] and return q=i+1. The left region is ≤pivot and the right region >pivot; neither side is necessarily sorted.

From the partition invariant to sorting correctness

1. **Initialization:** i=p−1,j=p makes both classified regions empty.
2. **Maintenance:** a small key expands ≤ after an exchange; a large key expands >. Either branch removes one unknown key and leaves the pivot fixed.
3. **Partition termination:** no unknown keys remain. The final exchange places the pivot at q. Exchanges preserve the input multiset.
4. **Recursive induction:** empty and singleton intervals are sorted. Both children exclude q and are strictly shorter. Sorting them and adjoining the pivot therefore sorts the whole interval.

The ordering relation alone is insufficient: the proof also needs **multiset preservation, strictly smaller recursive intervals**, and sorted children upon return.

Complete partition trace

Let A=[9,3,7,1,8,2,6],p=1,r=7,pivot=6, initially i=0.

j	Test	New i	Array
1	9>6	0	[9,3,7,1,8,2,6]
2	3≤6	1	[3,9,7,1,8,2,6], swap 1,2
3	7>6	1	[3,9,7,1,8,2,6]
4	1≤6	2	[3,1,7,9,8,2,6], swap 2,4
5	8>6	2	[3,1,7,9,8,2,6]
6	2≤6	3	[3,1,2,9,8,7,6], swap 3,6
End	place pivot at i+1	q=4	[3,1,2,6,8,7,9], swap 4,7

Left [3,1,2] is ≤6 and right [8,7,9] is >6; neither is sorted internally. Partition performs exactly r−p comparisons, Θ(r−p+1) time, and O(1) extra space.

Worst-case time: a family attaining quadratic work

Sorted distinct input with a fixed final pivot always chooses the maximum. The two child lengths are n−1 and 0.

Current length	Key comparisons	Next nonempty length
n	n−1	n−1
n−1	n−2	n−2
…	…	…
2	1	1

The sum is $n(n-1)/2$, proving a worst-case Ω(n^2) lower bound for an entire input family, rather than extrapolating from one measurement.

The quadratic upper bound must cover every split

Let C(n) be the worst comparison count. Child sizes satisfy a+b=n−1. Induction on smaller sizes gives

$$C(a) + C(b) + (n-1) \le \frac{a(a-1)}{2} + \frac{b(b-1)}{2} + n - 1 = \frac{n(n-1)}{2} - ab \le \frac{n(n-1)}{2}.$$

With C(0)=C(1)=0, this covers all splits. Equality holds when a=0 or b=0. Each pivot leaves recursion, so there are O(n) calls. Total time is O(n+C); the matching family proves **worst-case Θ(n²)**.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Balanced splits: count the comparison work by level

Take n=15 and always choose the true median. Count only Lomuto key comparisons; singletons do not call partition.

Depth d	Subproblems	Size each	Comparisons at level
0	1	15	14
1	2	7	2×6=12
2	4	3	4×2=8
3	8	1	0

Total: 34 comparisons, versus 105 for endpoint splits on 15 keys. Partition still scans linearly; the difference is how much work survives at each depth.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A fixed split ratio also gives logarithmic depth

If the larger child always has size at most ρ times the parent size, for a fixed ρ<1, the longest branch has O(log n) levels. Intervals at one depth are disjoint, giving at most n scanned keys per level and O(n log n) total work. A fixed 1:9 split suffices.

For perfectly balanced $n = 2^k - 1$, depth d has 2^d subproblems of length $2^{k-d} - 1$. Its comparison work is

$$2^d(2^{k-d} - 2) = n + 1 - 2^{d+1}, \qquad 0 \le d \le k - 2.$$

The first roughly k/2 levels each require Θ(n) comparisons, proving Ω(n log n) for that execution and hence Θ(n log n) overall. This lower bound counts actual recursive work; a comparison-sorting worst-input lower bound does not apply to every individual input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Duplicate keys defeat this two-way partition

On [5,5,5,5,5], every ≤ test succeeds and q=r, leaving n−1 keys on the left. Randomly exchanging a pivot to the end still exchanges a 5, giving ten comparisons here and n(n−1)/2 generally.

The following O(n log n) random-pivot analysis therefore assumes distinct keys. For many duplicates, use <,=,> regions and never recurse on the equal region. Chapter 9 gives full interval boundaries and rank conversions.

Random pivots do not remove this duplicate-key premise. Stability is separate: long-distance swaps can change equal-key identity order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recursion space and the smaller-side transformation

Two recursive calls can produce n stack levels. In-place partitioning does not make the whole algorithm unconditionally constant-space.

```
while p<r:
    q=partition(A,p,r)
    if q-p < r-q:
        quicksort-small-stack(A,p,q-1)
        p=q+1
    else:
        quicksort-small-stack(A,q+1,r)
        r=q-1
```

The actual recursive call has size at most (n−1)/2. The larger side continues in the current frame through a while loop. Thus sizes halve along a frame chain, giving worst-case O(log(n+1)) stack. Every partition still executes, and worst time remains $\Theta(n^2)$; reducing stack does not prove O(n log n) time.

7.3 A randomized version of quicksort

On current A[p..r], choose uniform k in p..r, swap A[k],A[r], then apply the same partition. Repeat uniform selection on every current child. With distinct keys, pivot rank is uniform in 1..n, so every fixed input order has the same expected analysis.

Expectation concerns internal random choices, not a uniform user-data assumption. An execution can keep selecting endpoints and cost $\Theta(n^2)$. The next proof establishes an expectation, not a per-execution or an unproved tail bound.

The ideal model requires uniform sampling conditional on current state. A fixed public pseudorandom sequence is a reproducible implementation choice whose adversarial guarantees need separate justification. A random API call by itself is not a proof of mathematical uniformity.

Which pair compares? Watch the first pivot within its rank interval

Name distinct keys by sorted order $z_1<\ldots<z_n$. Let $X_{ij}=1$ when z_i and z_j compare in any partition. A pair compares at most once: one endpoint is the pivot and then leaves recursion.

Consider $Z_{ij}=\{z_i,\ldots,z_j\}$. An outside pivot keeps this entire interval in one child. Until the first pivot from the interval appears, both endpoints therefore remain together.

Sorted ranks for analysis: $Z_{2,5}$ = {2,3,4,5}; not an array snapshot



Pivot 7 first: ranks 2..5 remain together in its left child; $X_{2,5}$ is undecided.

First within interval: endpoint 2 (or 5)



This partition compares 2 and 5 → $X_{2,5}=1$

First within interval: interior 3 (or 4)



2 left, 5 right → never compare, $X_{2,5}=0$

Four interval keys are equally likely to be first; two are endpoints. $P(X_{2,5}=1)=2/4=1/2$.

Endpoint first and interior first give different outcomes

For z_2,z_5, ignore outside pivots and inspect the first chosen key from {2,3,4,5}.

First interval pivot	What happens to 2 and 5?	$X_{2,5}$
2	This partition directly compares them	1
3	They enter separate children and never meet again	0
4	They enter separate children and never meet again	0
5	This partition directly compares them	1

Each current subproblem chooses a uniform pivot; outside choices do not distinguish the four interval keys. Each is therefore equally likely to be first within the interval. In general,

$$P(X_{ij}=1)=\frac{1}{j-i+1}+\frac{1}{j-i+1}=\frac{2}{j-i+1}.$$

The denominator is the interval length j−i+1, **not the full input length n**. The endpoint need not be the first pivot of the entire execution.

Expected comparisons: sum by rank-interval length

The total is $C = \sum_{i<j} X_{ij}$. Linearity of expectation needs no independence. Set k=j−i+1; exactly n+1−k rank intervals have length k. Thus

$$\begin{aligned} E[C] &= \sum_{i<j} \frac{2}{j-i+1} = 2\sum_{k=2}^{n} \frac{n+1-k}{k} \\ &= 2(n+1)(H_n - 1) - 2(n-1) \\ &= 2(n+1)H_n - 4n, \quad H_n = \sum_{k=1}^{n} \frac{1}{k}. \end{aligned}$$

The bounds $\ln(n+1) \le H_n \le 1 + \ln n$ give Θ(n log n) expected comparisons. Loop work has the same order; adding O(n) call overhead gives Θ(n log n) expected time.

This uses linearity alone. Comparison events are dependent, so variance or tail formulas for independent events do not automatically apply.

The same name partition can hide a different contract

Compare the contracts specified in these two lessons:

Item	Here: Lomuto	Data Structures Chapter 7: three-way
Indices and interval	1-based inclusive A[p..r]	0-based half-open A[l:r]
Return value	One pivot position q	Equal interval [lt,gt)
Recursive children	A[p..q−1], A[q+1..r]	A[l:lt], A[gt:r]
Equal keys	Non-pivot equal keys may recurse	The whole equal region leaves recursion

Our exact formula counts only `A[j] <= pivot`, excluding index tests, exchanges, and random generation. A three-way implementation testing < and then > may use two Boolean comparisons per classification. **Derive its exact constants separately** instead of transferring a count based on a function name.

With distinct keys and uniform pivots, both versions take expected Θ(n log n) time. All-equal input still costs $\Theta(n^2)$ with two-way Lomuto, but one three-way partition finishes in Θ(n).

All comparison probabilities for four ranks

Pair	Interval size	Probability
(1,2),(2,3),(3,4)	2	1 each
(1,3),(2,4)	3	2/3 each
(1,4)	4	1/2

The sum is 3+4/3+1/2=29/6, matching $2 \cdot 5 \cdot H_4 - 16$. Endpoints 1,4 separate if pivot 2 or 3 comes first and compare if 1 or 4 comes first. Each first choice has probability 1/4.

Adjacent ranks always comparing does not imply every pair compares. Nor is 29/6 the cost of one execution; four keys can require six comparisons in the worst case. It is an average over internal random choices.

Further reading: CLRS Chapter 7, § 7.1 for the partition invariant and § 7.4.2 for the random-pivot comparison analysis.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study